Les Plus Beaux Logis de Paris
Partie 1

Les Plus Beaux Logis de Paris

Titre du projet : Réalisez une analyse de données avec Python

Prénom : Alexander

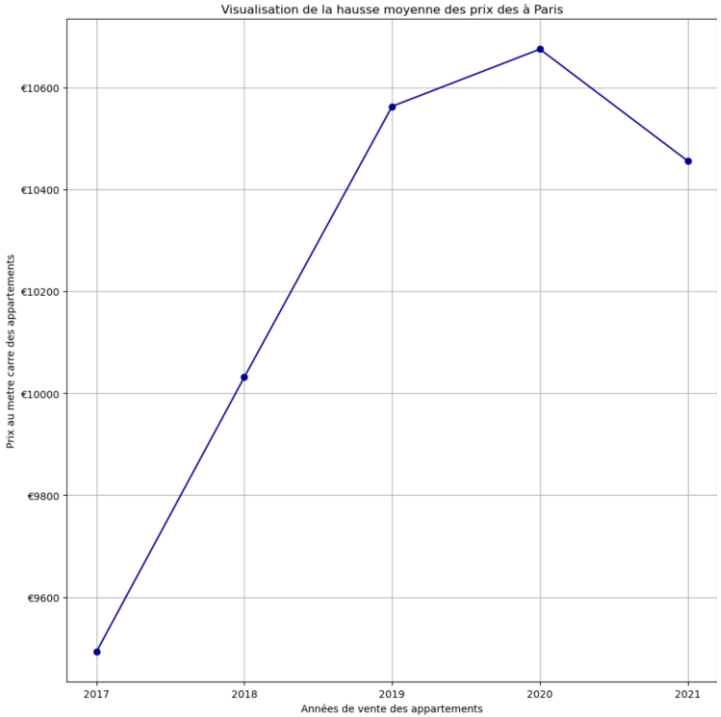
Nom : Lawson

Date : 02/10/25

I. Analyse du marché de l'immobilier

Evolution du prix au metre carré des appartements dans Paris

	annee	prix_moyen_vente	surface_moyenne
0	2017	427963.238883	44.627737
1	2018	448055.762541	44.271671
2	2019	461950.057064	43.361645
3	2020	462383.253604	42.900654
4	2021	458771.507203	43.479864



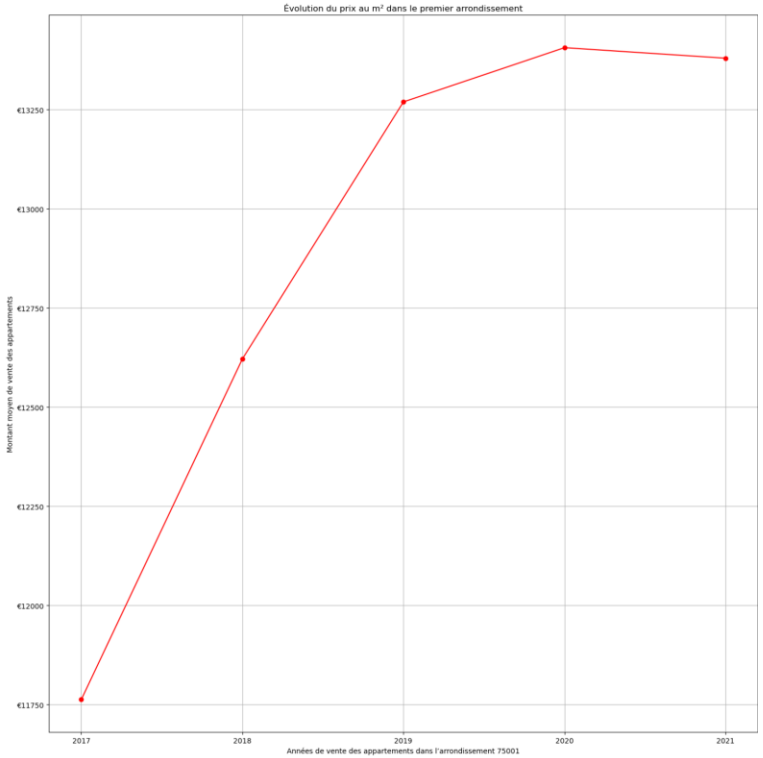
On constate que la moyenne des prix de vente

des appartements à Paris augmente au fils des années

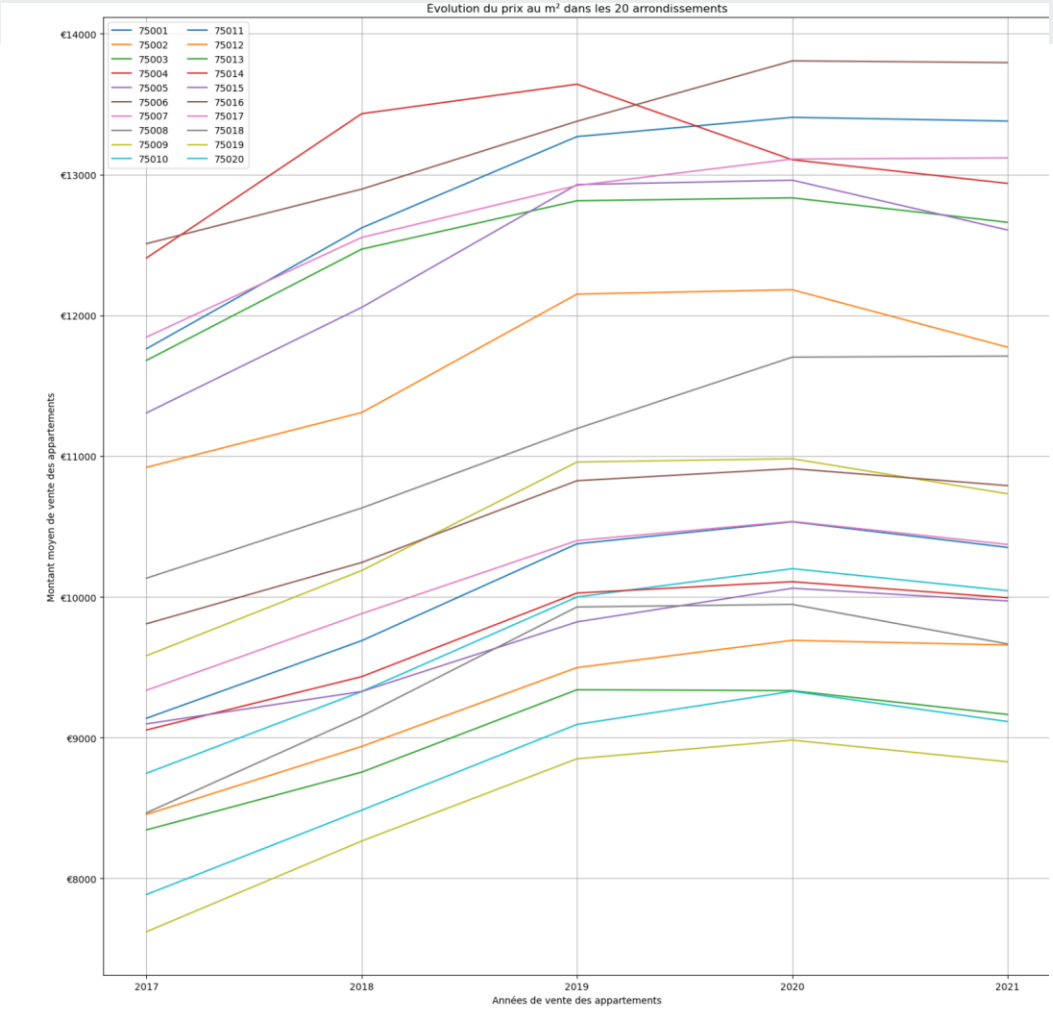
Différences de prix au metre carré entre les arrondissements

On voit que le 1er arrondissement suit la tendance de la moyenne des autres appartements avec une montée du prix du mètre carré puis une baisse à partir de 2020

Nous allons ajouter à cette visualisation tous les autres arrondissements avec une couleur différente.



- On observe que globalement après 2020 le prix du m2 fini par baisser après une augmentation de 3 ans

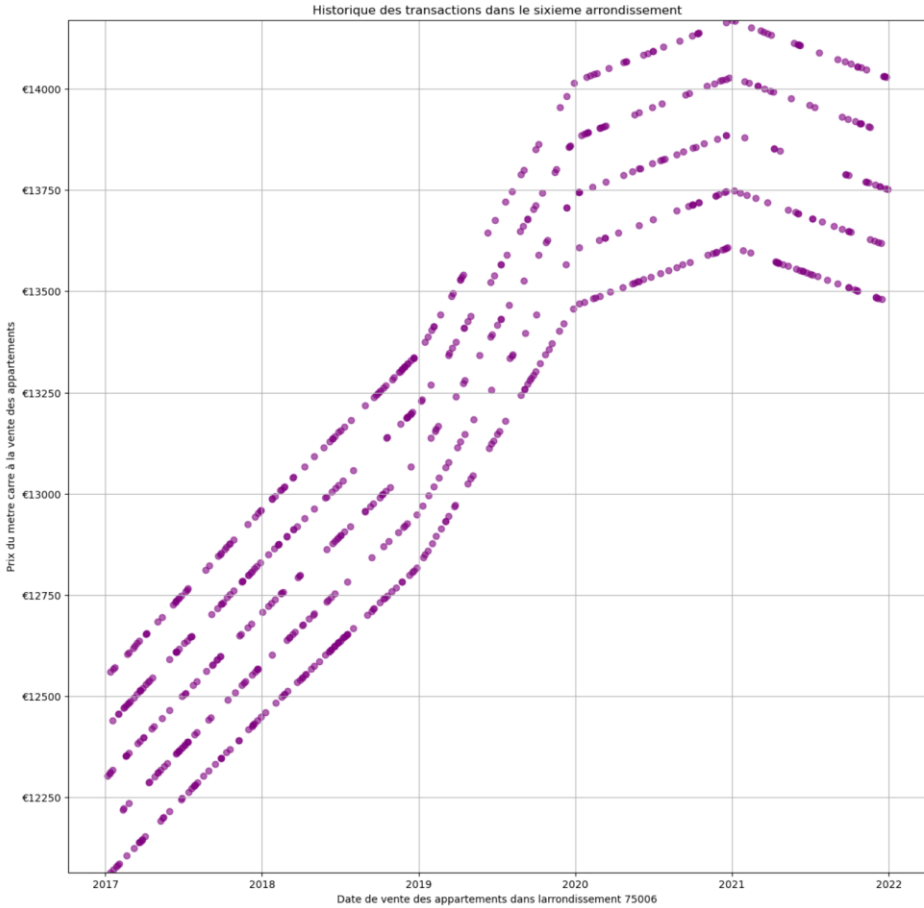


Prix au mètre carré par transaction dans le 6 ème arrondissement

On observe 709 transactions dans
le 6eme arrondissement

Le scatter plot montre une concentration plus faible de transaction à partir de 2020 et que le prix tourne autour de 14000 euros par m2

Pour nous rassurer sur la relation entre les données nous allons utiliser un test statistique de corrélation.



- ***Vérification de la relation entre le prix au m² et la date dans le 6ème arrondissement par le calcul du coefficient de corrélation de Pearson***

Pour cette distribution nous allons calculer le coefficient de corrélation de Pearson.
La relation entre le prix au mètre carré et la date n'est pas forcément linéaire mais elle s'en rapproche suffisamment pour que cette analyse soit pertinente .
Ce qui nous intéresse c'est de prouver que le temps a une influence sur le prix.

```python
# Calcul de la corrélation de Pearson et de la p-value
correlation, p_value = pearsonr(appart_75006['annee'], appart_75006['prix_m2'])

# Affichage
print(f"Coefficient de corrélation : {correlation:.4f}")
print(f"P-value : {p_value:.4e}")

# Interprétation simple
if p_value < 0.05:
    print("La corrélation est statistiquement significative.")
else:
    print("La corrélation n'est pas significative.")
```

```
Coefficient de corrélation : 0.8916
P-value : 1.7235e-245
La corrélation est statistiquement significative.
```

Un coefficient de corrélation de Pearson de 0.8916 indique une très forte corrélation positive entre deux variables,Cela signifie que plus le temps avance, plus le prix au m² augmente dans le 6e arrondissement.

- ***Vérification de la relation entre la valeur foncière et la surface***

Le coefficient de corrélation est de 0.9978 avec une pvalue de 0 .
La relation est donc confirmée.
Regardons maintenant si le type de bien à une influence sur le prix au mètre carré également.

```python
# Calcul de la corrélationde Pearson et de la p-value
correlation, p_value = pearsonr(appart_75006['valeur_fonciere'], appart_75006['surface_reelle'])

# Affichage
print(f"Coefficient de corrélation : {correlation:.4f}")
print(f"P-value : {p_value:.4e}")

# Interprétation simple
if p_value < 0.05:
    print("La corrélation est statistiquement significative.")
else:
    print("La corrélation n'est pas significative.")
```

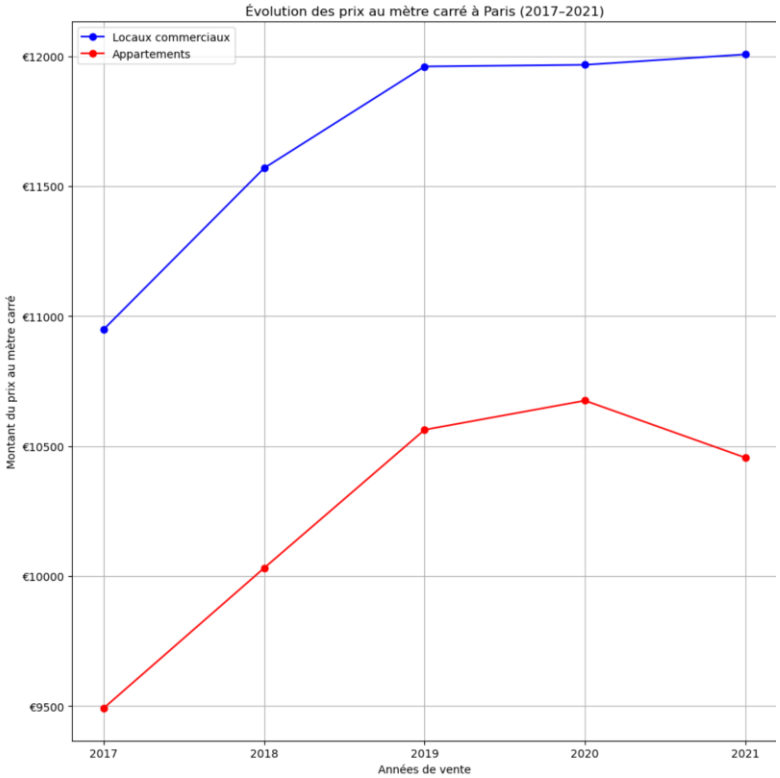
```
Coefficient de corrélation : 0.9978
P-value : 0.0000e+00
La corrélation est statistiquement significative.
```

Analyse des locaux industriels, commerciaux et assimilés

Le prix au mètre carré des locaux commerciaux est largement supérieur à celui des appartements

Après ces analyses, nous pouvons conclure que les dimensions à utiliser pour prédire le prix au m² sont :

- la surface du bien immobilier,
- la date considérée,
- la localisation (code_postal),
- le type de bien.



II. Méthodologie suivie

Dans cette section nous allons maintenant entraîner un algorithme à prédire la valeur fonciere d'un bien immobilier. Pour cela nous allons utiliser l'algorithme de régression linéaire.

On commence par préparer nos données en transformant les colonnes catégoriques du code postal et du type de local grâce au one hot encoder (sklearn) / get_dummies (pandas)

```
# 1. Conversion date en secondes
immobilier['timestamp_sec'] = immobilier['date_mutation'].astype('int64') // 10**9
```

```
# Encodage des variables catégorielles
immobilier_encoded = pd.get_dummies(immobilier, columns=['code_postal', 'type_local'])

# Définition des variables X et y
X = immobilier_encoded.drop(columns=['valeur_fonciere', 'date_mutation', 'adresse_nom_voie', 'nom_commune', 'adresse_numero'
y = immobilier_encoded['valeur_fonciere']
```

- On utilise le train_test_split pour prélever un tiers de nos données (33%) et les garder de côté. Nous allons entraîner notre algorithme sur le reste des données et puis mesurer notre erreur moyenne en pourcentage de la valeur foncière

```python
# On sépare le jeu de données entre echantillons d'apprentissage et de test
# La valeur y à trouver est la valeur foncière
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

```python
# 5. Normalisation (optionnelle mais utile pour LinearRegression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Entraînement
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
```

```python
# Évaluation sur le jeu de test
y_pred = lr.predict(X_test_scaled)
erreur_moyenne = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
print(f"Erreur moyenne : {erreur_moyenne:.2f} %")
```

```
Erreur moyenne : 9.53 %
```

Notre algorithme fait donc 9.53 % d'erreur en moyenne sur la prédiction de la valeur foncière.

Mes conclusions sur ce résultat et comment j'aurais pu aller plus loin :

-Tester d’autres algorithmes comme Random Forest ou Gradient Boosting

-Retirer les outliers qui faussent la régression

Nous avons récupéré le fichier avec le portefeuille des actifs de la société. Nous allons l'importer puis effectuer la prédiction et statuer sur la branche qui, selon notre prédiction, aura le plus de valeur à la date demandée c'est à dire au 31 décembre 2022.

Nous avons la liste des biens immobiliers de l'entreprise. Pour effectuer une prédiction, nous devons mettre ce fichier au même format que le dataframe que nous avons utilisé lors de l'entraînement de l'algorithme.

```
#On réutilise les mêmes fonctions pour faire le one hot encoding des variables catégorielles

portefeuille['surface_reelle'] = portefeuille['surface_reelle_bati']  # clé !
# Encodage des variables catégorielles
portefeuille_encoded = pd.get_dummies(portefeuille, columns=['code_postal', 'type_local'])

# On aligne les colonnes avec celles du modèle
missing_cols = set(X_train.columns) - set(portefeuille_encoded.columns)
for col in missing_cols:
    portefeuille_encoded[col] = 0  # Ajout des colonnes manquantes avec des zéros

# Réorganisation des colonnes
portefeuille_encoded = portefeuille_encoded[X_train.columns]
```

III. Résultat des prédictions

```python
#Valorisation du portefeuille sur le segment des particuliers
print(f"La valorisation du segment particulier est (en millions d'euros) : {valeur_particuliers:.2f}")
```

```
La valorisation du segment particulier est (en millions d'euros) : 71.05
```

```python
#Valorisation du portefeuille sur le segment corporate
print(f"La valorisation du segment corporate est (en millions d'euros) : {valeur_corporate:.2f}")
```

```
La valorisation du segment corporate est (en millions d'euros) : 97.97
```

Mes conclusions sur le segment avec la plus grande valorisation et sur les limites de cette estimation :

- La différence de valorisation est de 27 millions entre les 2 segments, celui du corporate est le plus élevé mais étant donné la marge d'erreur de 9.53%, cette dernière ne peut pas etre ignorée.
- Le modèle a été entrainé sur des données entre 2017 et 2021 qui rentre dans un contexte économique inhabituel, le modèle ne prend pas en compte ces aspects dans la prédiction des données.

Les Plus Beaux Logis de Paris
Partie 2

I. Méthodologie suivie

Dans cette partie nous allons labelliser automatiquement les biens immobiliers comme étant :

- soit des Appartements
- soit des Local industriel. commercial ou assimilé Pour cela nous allons utiliser l'algorithme du KMeans sur le jeu de données partagé par l'entreprise.

Pour que l'algorithme fonctionne, il faut que nous préparions les données en supprimant les dimensions inutiles et en nous concentrant sur le facteur discriminant entre les appartements et les locaux commerciaux : la différence dans le prix au mètre carré tel que nous l'avons vu avant.

On applique les transformations nécessaires. Tout d'abord nous allons calculer le prix au mètre carré en divisant la valeur foncière par la surface. Ensuite nous allons retirer ces colonnes car nous avons déjà l'information qu'elles contiennent dans la dimension prix au mètre carré désormais.

```
# Calcul du prix au m²
df_test['prix_m2'] = df_test['valeur_fonciere'] / df_test['surface_reelle']

# Colonnes à supprimer (inutiles pour KMeans)
df_test_clean = df_test.drop(columns=['valeur_fonciere', 'surface_reelle'])
```

Nous observons dans les données que nous avons des valeurs différentes de prix au mètre carré pour un même arrondissement (ici le 19ème arrondissement). Il se peut fort que cela soit notre dimension à utiliser pour attribuer les prix au mètre carré les plus élévé dans un département aux locaux commerciaux, et les prix les plus bas aux appartements.

Pour effectuer cette opération, nous allons utiliser l'algorithme du Kmeans qui va rechercher 2 centroïdes à travers les données.

```
X_scaled = scaler.fit_transform(df_test_clean[['prix_m2']])
```

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, random_state=0, n_init=15)
df_test_clean['cluster'] = kmeans.fit_predict(X_scaled)

print("Centroïdes normalisés :", kmeans.cluster_centers_)
```

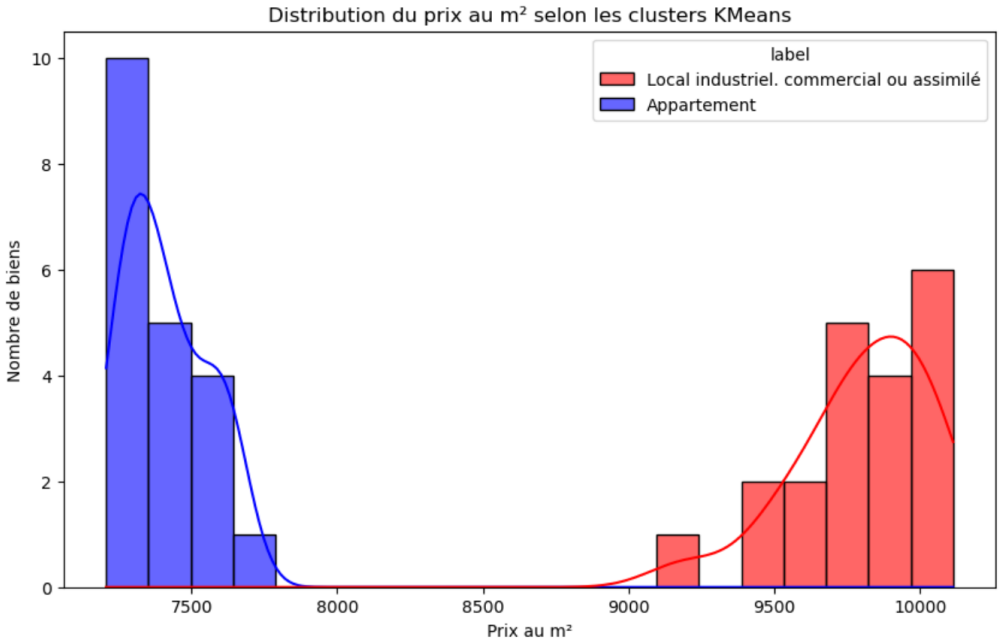
```
Centroïdes normalisés : [[-0.98795737]
 [ 0.98795737]]
```

```
# On vérifie les données de la prédiction
# Prix moyen par cluster
prix_moyen_clusters = df_test_clean.groupby('cluster')['prix_m2'].mean()
print(prix_moyen_clusters)

# Mapping automatique
if prix_moyen_clusters[0] > prix_moyen_clusters[1]:
    mapping = {0: "Local industriel. commercial ou assimilé", 1: "Appartement"}
else:
    mapping = {1: "Local industriel. commercial ou assimilé", 0: "Appartement"}

df_test_clean['label'] = df_test_clean['cluster'].map(mapping)

df_test_clean[['code_postal', 'prix_m2', 'label']].head(10)
```

```
cluster
0    7408.775030
1    9806.924674
Name: prix_m2, dtype: float64
```

II. Résultat de la classification

Nous avons obtenu notre prédiction. Nous pouvons changer les labels et remplacer les valeurs à 0 par Local industriel. commercial ou assimilé et les valeurs à +1 par Appartement.



Conclusion

Le prix au m² est bien le facteur discriminant clé entre appartements et locaux commerciaux.
Le clustering confirme que, même sans labels, on peut séparer les biens en deux catégories cohérentes.

- Les limites de l'exercice :

KMeans : ne connaît pas les vrais labels → nécessite une interprétation humaine. Sensibilité aux outliers
Quelques valeurs extrêmes peuvent déplacer fortement les centroïdes et fausser la classification. Sans normalisation, le kmean serait sensible à l'échelle et donc peut biaise les résultats